

Exercice SQL - CORRIGÉ

Gestion de Bibliothèque

ISTA Mirleft

Cas d'étude : Bibliothèque de l'ISTA

Une bibliothèque prête des livres aux étudiants. Chaque emprunt est géré par un bibliothécaire.

Règles :

- Un étudiant peut emprunter plusieurs livres
- Un livre peut être emprunté par plusieurs étudiants (à différents moments)
- Chaque emprunt a une date d'emprunt, une date de retour prévue et un statut

Schéma relationnel :

```
ETUDIANTS (id_etudiant, nom, prenom, filiere, niveau)
LIVRES (isbn, titre, auteur, categorie)
BIBLIOTHECAIRES (id_biblio, nom, prenom, email)
EMPRUNTS (id_emprunt, #id_etudiant, #isbn, #id_biblio,
           date_emprunt, date_retour_prevue, statut)
```

(# = clé étrangère)

1 Création de la Base de Données

1. Créez une base de données nommée `bibliotheque`.

Solution :

```
CREATE DATABASE bibliotheque;
```

2. Sélectionnez cette base de données.

Solution :

```
USE bibliotheque;
```

2 Création des Tables

3. Créez la table `etudiants` :

- `id_etudiant` : INT, clé primaire, auto-incrémenté
- `nom` : VARCHAR(50), obligatoire
- `prenom` : VARCHAR(50), obligatoire
- `filiere` : VARCHAR(50)
- `niveau` : VARCHAR(20)

Solution :

```
CREATE TABLE etudiants (  
    id_etudiant INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    prenom VARCHAR(50) NOT NULL,  
    filiere VARCHAR(50),  
    niveau VARCHAR(20)  
);
```

4. Créez la table livres :

- isbn : VARCHAR(20), clé primaire
- titre : VARCHAR(100), obligatoire
- auteur : VARCHAR(100)
- categorie : VARCHAR(50)

Solution :

```
CREATE TABLE livres (  
    isbn VARCHAR(20) PRIMARY KEY,  
    titre VARCHAR(100) NOT NULL,  
    auteur VARCHAR(100),  
    categorie VARCHAR(50)  
);
```

5. Créez la table bibliothecaires :

- id_biblio : INT, clé primaire, auto-incrémenté
- nom : VARCHAR(50), obligatoire
- prenom : VARCHAR(50), obligatoire
- email : VARCHAR(100), unique

Solution :

```
CREATE TABLE bibliothecaires (  
    id_biblio INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    prenom VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE  
);
```

6. Créez la table emprunts :

- id_emprunt : INT, clé primaire, auto-incrémenté
- id_etudiant : INT, clé étrangère vers etudiants
- isbn : VARCHAR(20), clé étrangère vers livres
- id_biblio : INT, clé étrangère vers bibliothecaires
- date_emprunt : DATE, obligatoire
- date_retour_prevue : DATE, obligatoire
- statut : VARCHAR(20)

Solution :

```
CREATE TABLE emprunts (  
    id_emprunt INT AUTO_INCREMENT PRIMARY KEY,  
    id_etudiant INT,  
    isbn VARCHAR(20),  
    id_biblio INT,  
    date_emprunt DATE NOT NULL,  
    date_retour_prevue DATE NOT NULL,  
    statut VARCHAR(20),  
    FOREIGN KEY (id_etudiant) REFERENCES etudiants(id_etudiant),
```

```
FOREIGN KEY (isbn) REFERENCES livres(isbn),
FOREIGN KEY (id_biblio) REFERENCES bibliothecaires(id_biblio)
);
```

7. Affichez la liste de toutes les tables créées.

Solution :

```
SHOW TABLES;
```

Résultat attendu :

```
+-----+
| Tables_in_bibliotheque |
+-----+
| bibliothecaires        |
| emprunts               |
| etudiants              |
| livres                 |
+-----+
```

3 Insertion de Données

8. Insérez 2 bibliothécaires :

```
('Alami', 'Fatima', 'f.alami@ista.ma')
('Bennani', 'Ahmed', 'a.bennani@ista.ma')
```

Solution :

```
INSERT INTO bibliothecaires (nom, prenom, email) VALUES
('Alami', 'Fatima', 'f.alami@ista.ma'),
('Bennani', 'Ahmed', 'a.bennani@ista.ma');
```

9. Insérez 4 étudiants :

```
('Idrissi', 'Hassan', 'Dev_Digital', '1ere_annee')
('Chafik', 'Amina', 'Dev_Digital', '2eme_annee')
('Tazi', 'Omar', 'Reseaux', '1ere_annee')
('El_Amri', 'Salma', 'Dev_Digital', '1ere_annee')
```

Solution :

```
INSERT INTO etudiants (nom, prenom, filiere, niveau) VALUES
('Idrissi', 'Hassan', 'Dev_Digital', '1ere_annee'),
('Chafik', 'Amina', 'Dev_Digital', '2eme_annee'),
('Tazi', 'Omar', 'Reseaux', '1ere_annee'),
('El_Amri', 'Salma', 'Dev_Digital', '1ere_annee');
```

10. Insérez 5 livres :

```
('978-01', 'HTML_et_CSS', 'Jon_Duckett', 'Web')
('978-02', 'JavaScript', 'David_Flanagan', 'Programmation')
('978-03', 'SQL_Facile', 'Chris_Fehily', 'Base_de_donnees')
('978-04', 'Reseaux', 'Andrew_Tanenbaum', 'Reseaux')
('978-05', 'Python', 'Mark_Lutz', 'Programmation')
```

Solution :

```
INSERT INTO livres (isbn, titre, auteur, categorie) VALUES
('978-01', 'HTML et CSS', 'Jon Duckett', 'Web'),
('978-02', 'JavaScript', 'David Flanagan', 'Programmation'),
('978-03', 'SQL Facile', 'Chris Fehily', 'Base de donnees'),
('978-04', 'Reseaux', 'Andrew Tanenbaum', 'Reseaux'),
('978-05', 'Python', 'Mark Lutz', 'Programmation');
```

11. Insérez 3 emprunts :

```
(1, '978-01', 1, '2024-01-15', '2024-02-15', 'En cours')
(2, '978-02', 1, '2024-01-20', '2024-02-20', 'En cours')
(3, '978-03', 2, '2024-01-10', '2024-02-10', 'Retourne')
```

Solution :

```
INSERT INTO emprunts (id_etudiant, isbn, id_biblio,
                      date_emprunt, date_retour_prevue, statut) VALUES
(1, '978-01', 1, '2024-01-15', '2024-02-15', 'En cours'),
(2, '978-02', 1, '2024-01-20', '2024-02-20', 'En cours'),
(3, '978-03', 2, '2024-01-10', '2024-02-10', 'Retourne');
```

4 Requêtes de Consultation

12. Affichez tous les étudiants.

Solution :

```
SELECT * FROM etudiants;
```

13. Affichez tous les livres de la catégorie 'Programmation'.

Solution :

```
SELECT * FROM livres WHERE categorie = 'Programmation';
```

Résultat attendu :

isbn	titre	auteur	categorie
978-02	JavaScript	David Flanagan	Programmation
978-05	Python	Mark Lutz	Programmation

14. Comptez le nombre total de livres.

Solution :

```
SELECT COUNT(*) AS nombre_livres FROM livres;
```

Résultat attendu :

nombre_livres
5

15. Affichez les emprunts avec le statut 'En cours'.

Solution :

```
SELECT * FROM emprunts WHERE statut = 'En_cours';
```

16. Affichez le nom de l'étudiant, le titre du livre et la date d'emprunt pour tous les emprunts. (Utilisez JOIN)

Solution :

```
SELECT e.nom, e.prenom, l.titre, emp.date_emprunt
FROM emprunts emp
INNER JOIN etudiants e ON emp.id_etudiant = e.id_etudiant
INNER JOIN livres l ON emp.isbn = l.isbn;
```

Résultat attendu :

nom	prenom	titre	date_emprunt
Idrissi	Hassan	HTML et CSS	2024-01-15
Chafik	Amina	JavaScript	2024-01-20
Tazi	Omar	SQL Facile	2024-01-10

17. Affichez le nombre d'emprunts par étudiant. (Utilisez GROUP BY)

Solution :

```
SELECT e.nom, e.prenom, COUNT(emp.id_emprunt) AS nombre_emprunts
FROM etudiants e
LEFT JOIN emprunts emp ON e.id_etudiant = emp.id_etudiant
GROUP BY e.id_etudiant, e.nom, e.prenom;
```

Résultat attendu :

nom	prenom	nombre_emprunts
Idrissi	Hassan	1
Chafik	Amina	1
Tazi	Omar	1
El Amri	Salma	0

5 Modification de Données

18. Changez le statut de l'emprunt numéro 1 en 'Retourne'.

Solution :

```
UPDATE emprunts
SET statut = 'Retourne'
WHERE id_emprunt = 1;
```

19. Modifiez la catégorie du livre '978-02' en 'Dev Web'.

Solution :

```
UPDATE livres
SET categorie = 'Dev_Web'
WHERE isbn = '978-02';
```

20. Changez l'email du bibliothécaire ayant l'id 1 en 'fatima.alami@ista.ma'.

Solution :

```
UPDATE bibliothecaires
SET email = 'fatima.alami@ista.ma'
WHERE id_biblio = 1;
```

6 Suppression de Données

21. Supprimez l'emprunt numéro 3.

Solution :

```
DELETE FROM emprunts WHERE id_emprunt = 3;
```

22. Supprimez le livre ayant l'isbn '978-05'.

Solution :

```
DELETE FROM livres WHERE isbn = '978-05';
```

23. Supprimez tous les emprunts de l'étudiant ayant l'id 2, puis supprimez cet étudiant.

Solution :

```
-- D'abord supprimer les emprunts
DELETE FROM emprunts WHERE id_etudiant = 2;

-- Ensuite supprimer l'etudiant
DELETE FROM etudiants WHERE id_etudiant = 2;
```

Note : Il faut d'abord supprimer les emprunts (qui contiennent la clé étrangère) avant de supprimer l'étudiant référencé.

7 Modification de Tables

24. Ajoutez une colonne email de type VARCHAR(100) à la table etudiants.

Solution :

```
ALTER TABLE etudiants ADD COLUMN email VARCHAR(100);
```

25. Supprimez la colonne email de la table etudiants.

Solution :

```
ALTER TABLE etudiants DROP COLUMN email;
```

26. Créez une table test avec 2 colonnes (id INT, nom VARCHAR(50)), puis supprimez-la.

Solution :

```
-- Creation de la table
CREATE TABLE test (
    id INT,
    nom VARCHAR(50)
);

-- Suppression de la table
DROP TABLE test;
```

Remarques Importantes

- **Clés étrangères** : Lors de la suppression, respectez l'ordre : supprimer d'abord les enregistrements enfants (avec clés étrangères) avant les parents.
- **Contraintes d'intégrité** : Les contraintes FOREIGN KEY garantissent la cohérence des données entre les tables.
- **JOIN** : Utilisez INNER JOIN pour les relations obligatoires et LEFT JOIN quand vous voulez inclure les lignes sans correspondance.
- **GROUP BY** : Toutes les colonnes du SELECT qui ne sont pas dans une fonction d'agrégation doivent être dans GROUP BY.